Apache Spark Programming

Deep Dive into Spark RDD Programming

---

1) What an RDD Really Represents (Deeper Meaning)

- An **RDD is not "data sitting in one place"** like a file/table/collection.
- It is a **logical abstraction** of data spread across many machines **plus the definition of how it should be processed**.
- When you work with an RDD:
    - You are **not manually manipulating bytes** on disk/memory.
    - You are **describing a distributed computation** that Spark will execute **when needed**.

RDD as a "Contract / Promise"

- Think of an RDD as a **promise**:
    - "This dataset exists"
    - "It is partitioned across the cluster"
    - "Spark knows exactly how each partition was created"
- The key value is **not the location of data**, but the **instructions (definition)** Spark has to build/transform it.
- This is the reason RDDs are **resilient**: Spark can recreate partitions using the recorded definition (lineage).

Parallelism by Design

- RDDs are divided into **partitions**.
- Each partition can be processed **independently** → Spark distributes work automatically across executors.
- Developer focus: **what should happen to the data**
- Spark handles: **where/how it happens** (execution, scheduling, recovery).

Shift in Programming Model

- Instead of "step-by-step execution logic," you:
    - Describe **transformations** over a distributed dataset
    - Spark manages **execution + optimization + recovery**
- This abstraction lets developers focus on **correctness and intent**.

---

2) Three Core Concepts Revisited: Resilient, Parallelism, Fault Tolerance

A) Resilient (What it really means in Spark)

- Resilient ≠ "backed up or duplicated"
- Resilient = **Spark can recover lost data automatically by recomputing it**
- The power comes from **lineage**:
    - Spark remembers **how the RDD was built** (sequence of transformations)
    - If partitions are lost → Spark recomputes **only missing partitions**, not the entire job

Why this works well

- **RDDs are immutable + deterministic**
    - Same transformations → same output
    - Makes recomputation safe and predictable
- Spark avoids the heavy overhead of default replication for RDD fault recovery.

Conceptual takeaway

- Spark protects **computation logic (lineage)** more than "protecting data copies."
- Failures are expected in clusters; lineage-based recompute keeps jobs moving.

Industry Example (E-commerce clickstream)

- Daily terabytes of logs processed for metrics (page views, clicks, funnels).
- If a node fails mid-job:
    - Spark uses lineage: "read logs → filter invalid → map to structured fields"
    - Recomputes only lost partitions
- No manual recovery logic needed; job completes as expected.

B) Parallelism (Built into the RDD model)

- Parallelism = big processing broken into **smaller independent work units** running simultaneously.
- Achieved via **RDD partitions**:
    - Each partition is a unit of work
    - Spark schedules tasks across executors in parallel

Key points

- Developer writes logic once; Spark runs it **many times** on different partitions.
- **Scales naturally**:
    - More data or more machines → Spark can increase partitions and parallel execution
    - Same code still works (horizontal scalability)

Industry Example (Telecom Call Detail Records)

- Millions of CDRs daily; sequential processing is too slow.
- Spark loads CDRs as RDD → split into thousands of partitions.
- Filtering/aggregations per region run in parallel across executors.
- Need faster? Add machines; Spark scales without rewriting logic.

C) Fault Tolerance (Keep processing even when cluster fails)

- Fault tolerance = Spark continues correctly even if:
    - machines crash
    - executors are lost
    - network issues occur
- Mechanism: **lineage-based recomputation**
    - Identify lost partitions
    - Recompute only those partitions
- Immutability ensures recomputation produces consistent results (no partial updates).

Industry Example (Financial risk analysis)

- Large transaction data processed for daily risk.
- Worker node fails → Spark:
    - detects failure
    - identifies lost partitions
    - recomputes missing partitions from lineage
- Job completes without restarting fully; no special failure-handling code.

---

3) Why RDDs Never Change (Immutability)

What “Immutable” means

- Once created, an RDD **cannot be modified**.
- Any “change” produces a **new RDD**.

Why Spark designed it this way

1) Parallel Safety

- Many tasks run concurrently on partitions.
- If data were mutable → race conditions, locking, coordination complexity.
- Immutability avoids:
    - two tasks updating the same data
    - inconsistent states

2) Fault Tolerance becomes simple

- Since RDDs don't change:
    - Spark can recompute partitions from lineage reliably
- If RDDs were mutable:
    - Spark would need to track every in-place update and replicate states

3) Enables optimization & predictable execution

- Spark can reorder/combine/pipeline transformations safely because upstream RDDs won't change.
- Supports optimized DAG construction and execution planning.

Industry Example (Retail purchase logs)

- Raw transactions RDD → filter invalid records
- Spark creates a **new filtered RDD**, original remains intact
- If a node fails, Spark can recompute partitions from the original + transformations.

---

4) SparkContext and SparkSession (Entry Points)

A) SparkContext

What it is

- Starting point of Spark apps (connection between application and cluster).
- Communicates with cluster manager, requests resources, coordinates distributed tasks.

What it does conceptually

- Acts as a **control center**:
    - knows cluster configuration and resources
    - manages RDD creation
    - decides partitioning
    - tracks RDD lineage (key for fault tolerance)

> In modern Spark, SparkContext often exists **behind SparkSession**, but still performs the same responsibilities.

Code: Creating SparkContext

```
from pyspark import SparkConf, SparkContext

conf = SparkConf() \
    .setAppName("MyFirstSparkApp") \
    .setMaster("local[*]")

sc = SparkContext(conf=conf)
```

Meaning of steps:

- `SparkConf()` → configuration object
- `.setAppName(...)` → visible in Spark UI/logs
- `.setMaster("local[*]")` → run locally using all CPU cores
- `SparkContext(...)` → starts Spark for the application, ready for RDD operations

Analogy (Manufacturing plant)

- SparkContext = **operations manager**
    - sets up factory floor (cluster environment)

- allocates machines/workers (resources/executors)
- splits production across lines (partitioning/task distribution)
- handles breakdowns (reschedules via lineage + scheduler)

B) SparkSession (Modern unified entry point)

What it is

- Recommended entry point for Spark:
  - RDDs (indirect)
  - DataFrames
  - SQL
  - Hive support
- Provides a single unified interface rather than multiple contexts.

What it does internally

- Wraps and manages:
  - SparkContext
  - SQL engine & catalog services
- Central place for configuration and reading data sources.

Code: Creating SparkSession (as provided)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MyFirstSparkApp") \
```

```
    .master("local[*]") \
    .getOrCreate()

sc = spark.sparkContext
```

Meaning of steps :

- `builder` → configure Spark
- `appName` → name shown in UI/logs
- `master("local[*]")` → local execution with all cores
- `getOrCreate()` → reuse existing or create a new session
- `spark.sparkContext` → access underlying SparkContext when needed for RDDs

---

5) Reading Data into RDDs

A) Reading from a file (`textFile`)

- Common for real datasets from disk/HDFS/S3 etc.
- `sc.textFile(path)`:
    - each line becomes one element in RDD
    - file is split into partitions automatically
- Important: **lazy evaluation**
    - reading/transforming doesn't execute until an action is called

Code flow from script

Read file

```
rdd = sc.textFile("path/to/your/file.txt")
```

Inspect first 5 lines (Action)

```
print(rdd.take(5))
```

Transformation (lazy)

```
filtered_rdd = rdd.filter(lambda line: line.strip() != "")
```

Action triggers execution

```
count = filtered_rdd.count()
print(f"Number of non-empty lines: {count}")
```

Key notes (as stated):

- File split into partitions → parallel tasks
- Original RDD unchanged (immutability)
- If partition lost → Spark recomputes from original source (fault tolerance)

B) Creating RDD from in-memory data (`parallelize`)

- Used when data already exists in program (lists/collections).
- Useful for:
    - testing
    - prototyping
    - small computations
- Supports `numSlices` to control partitions (parallelism control).

Code example :-

```
from pyspark import SparkConf, SparkContext

conf =
SparkConf().setAppName("ParallelizeExample").setMaster("local[*]")
sc = SparkContext(conf=conf)

data = [1, 2, 3, 4, 5]

rdd = sc.parallelize(data, numSlices=2)

squared_rdd = rdd.map(lambda x: x ** 2)

print(squared_rdd.collect())  # Output: [1, 4, 9, 16, 25]
```

Highlights from explanation:

- `parallelize` → distributes elements across partitions
- `numSlices` controls number of partitions
- Transformations still lazy; `collect()` triggers execution
- Spark handles:
    - parallel execution
    - fault recovery
    - combining results to driver

---

6) RDD Transformations

What transformations are

- Operations applied to an RDD that produce a **new RDD** (immutability).
- They define a **logical description** of processing.
- Spark records transformations in the **DAG** to optimize later.
- Execution is **lazy** until an action is called.

Key points stated

- Transformations = “what to do”
- Spark manages:
    - scheduling
    - partitioning
    - fault recovery

Types of Transformations

A) Narrow Transformations

Definition:

- Each output partition depends on **exactly one** input partition.
- No shuffle required (no network movement).
- Fast, scalable, supports pipelining.

Characteristics

- Partition-local processing
- No inter-partition communication

- Can pipeline multiple narrow ops in same stage

Examples

- `map()`, `filter()`, `flatMap()`, `mapPartitions()`, `union()`

Industry analogy

- Factory conveyor belts: each worker works only on their belt; no moving items to other belts.

B) Wide Transformations

Definition :

- Each output partition depends on **multiple** input partitions.
- Requires **shuffle** (network + disk overhead).
- Breaks pipelining and creates new stages in DAG.

Characteristics

- Cross-partition dependency
- Shuffle mandatory (expensive)
- Higher latency but essential for joins/aggregations

Examples

- `groupByKey()`, `reduceByKey()`, `join()`, `cogroup()`, `distinct()`

Industry analogy

- Retail warehouses: items moved between warehouses so all items for an order come together (shuffle).

Narrow vs Wide Summary

- Narrow:
    - no shuffle
    - fast
    - pipelined within stage
- Wide:
    - shuffle required
    - slower due to network/disk
    - creates stage boundaries in DAG
    - needed for grouping/join/dedup

---

7) RDD Actions in Spark

What an Action is

- Actions are the point where Spark stops “planning” and starts “executing.”
- Without an action:
    - no computation happens
    - Spark stays declarative (just building a plan)

What actions do

- Trigger execution of all previous transformations
- Return result to driver OR write to external storage
- Spark then:
    - analyzes DAG
    - breaks into stages
    - runs tasks in parallel
    - applies lineage-based recovery if failures happen

Common Actions

- `count()` → number of elements
- `collect()` → bring all elements to driver (only safe for small data)
- `take(n)` → first n elements (safer than collect)
- `first()` → first element
- `reduce()` → aggregate into single result
- `foreach()` → run function for side effects
- `saveAsTextFile()` → write output (1 file per partition)

Internal trigger point

When an action occurs:

- CPU/network/disk get used
- executors start working
- Spark executes the pipeline defined so far

Industry analogy

- Planning a corporate report (transformations) vs publishing/sending it (action). Nothing is delivered until “publish.”

Interview Questions & Discussion Points

- What does an RDD represent “logically” (promise/contract concept)?
- How does Spark achieve resilience without default replication for RDDs?
- Explain lineage and how it supports recovery.
- Why are RDDs immutable? How does immutability help parallelism and fault tolerance?
- SparkContext vs SparkSession: purpose and relationship.
- Explain lazy evaluation with transformations/actions.
- Difference between narrow and wide transformations; why shuffles are expensive.
- Why is `reduceByKey` generally preferred over `groupByKey` (script calls out efficiency around wide operations and shuffles).
- When is `collect()` dangerous and why (brings everything to driver)?
- How does partitioning relate to parallelism and performance?

Recap of Learning Objectives

- RDDs are **logical distributed computation definitions**, not just stored data.
- **Resilience + fault tolerance** come from **lineage + immutability** (recompute missing partitions).

- **Parallelism** is built-in via partitions; Spark handles execution distribution.
- Spark programming starts via **SparkContext** (or **SparkSession** in modern apps).
- Data ingestion commonly happens through **textFile** (external) and **parallelize** (in-memory).
- Transformations are **lazy**, actions **trigger execution**.
- Understanding **narrow vs wide** transformations is key for performance awareness (shuffle boundaries).